

COMP1204: Data Management

Coursework One: Hurricane Monitoring

Ash-Hab Abbasi
34039414

March 16, 2023

1 Introduction

This is a report on the COMP1204 Data Management coursework which was primarily about developing a script that collects and processes data on hurricanes. The script takes in data from KML files and produces a CSV file that contains the relevant data needed on the storms. Git was used to managing the repository of the code and this report, as well as to handle any conflicts, including the one caused by the conflict-script.sh part of the coursework.

2 Create CSV Script

The create_csv.sh bash script file takes in two arguments, an input (KML file) and an output (CSV file). The KML file converts the hurricane data to a CSV file which contains only the relevant information required for plotting. While looking through the KML files, I noticed that all the relevant data needed were always under the 'CDATA' tag, so my script uses 'grep' to find all the lines in the input file that contains the string 'CDATA', extracts the line numbers (excluding the first one as the first appearance of CDATA doesn't include the necessary information), then a loop is used to iterate over each line number and extract the information using 'sed' and 'cut', it then outputs that information into the output csv file.

2.1 The code WITH comments:

```
#!/bin/bash

#first argument taken in is the input KML file
#second argument taken in is the output file
inputF=$1
outputF=$2

echo "Converting_${inputF}_->_${outputF}_..."

#writes the line into the output file, replaces all of the current output file
echo "Timestamp, Latitude, Longitude, MinSeaLevelPressure, MaxIntensity" >
    $outputF

#gets all line numbers where 'CDATA' is present (in the input KML file),
#excluding the first one
CDATALines=$(grep -n CDATA $inputF | cut -d':' -f1 | tail -n+2)

#for every line number where 'CDATA' is present...
for lineNum in $CDATALines; do
```

```

#The timestamp is located three lines below where CDATA's found
((lineNum+=3))

#gets the timestamp and puts it in a variable, clears out the unnecessary
information
timestamp=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g')

#The latitude/longitude is located two lines below the previous
((lineNum+=2))

#gets the latitude/longitude and puts it in a variable, clears out the
unnecessary information
latitudelongitude=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g' | sed
's/N, /N,/g')

#The minimum sea level pressure is located two lines below the previous
((lineNum+=2))

#gets the minimum sea level pressure and puts it in a variable, clears out the
unnecessary information
minsealevelpressure=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g' |
cut -d ';' -f1)

#The MaxIntensity is located two lines below the previous
((lineNum+=2))

#gets the maximum Intensity and puts it in a variable, clears out the
unnecessary information
maxintensity=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g' | cut -d ';'
-f1)

#appends all the gathered data together on a new line into the output file
echo "$timestamp,$latitudelongitude,$minsealevelpressure,$maxintensity" >>
$outputF

done

echo "Done!"

create_csv.sh bash script file (commented)

```

2.2 The code without comments:

```

#!/bin/bash

inputF=$1
outputF=$2

echo "Converting $inputF to $outputF..."
echo "Timestamp, Latitude, Longitude, MinSeaLevelPressure, MaxIntensity" >
$outputF

CDATAlines=$(grep -n CDATA $inputF | cut -d ':' -f1 | tail -n+2)

```

```

for lineNum in $CDATALines; do
((lineNum+=3))
timestamp=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g')
((lineNum+=2))
latitudelongitude=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g' | sed
's/N, /N,/g')
((lineNum+=2))
minsealevelpressure=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g' |
cut -d ' '; ' -f1)
((lineNum+=2))
maxintensity=$(sed "${lineNum}q;d" $inputF | sed 's/\s*<[^>]*>//g' | cut -d ' ';
' -f1)

echo "$timestamp,$latitudelongitude,$minsealevelpressure,$maxintensity" >>
$outputF

done

echo "Done!"
create_csv.sh bash script file (no comments)

```

3 Storm Plots

The create_map_plot.sh script produces images from the csv files generated by the create_csv.sh script. These plots show the locations of the storms. Below are 3 plots out of the 5 KML files that were given.

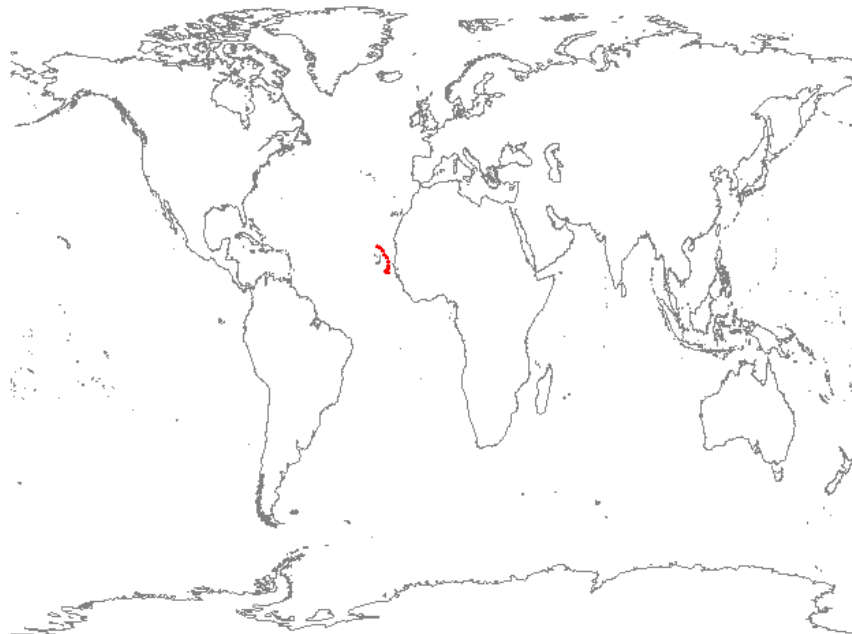


Figure 1: KML file: al102020.kml

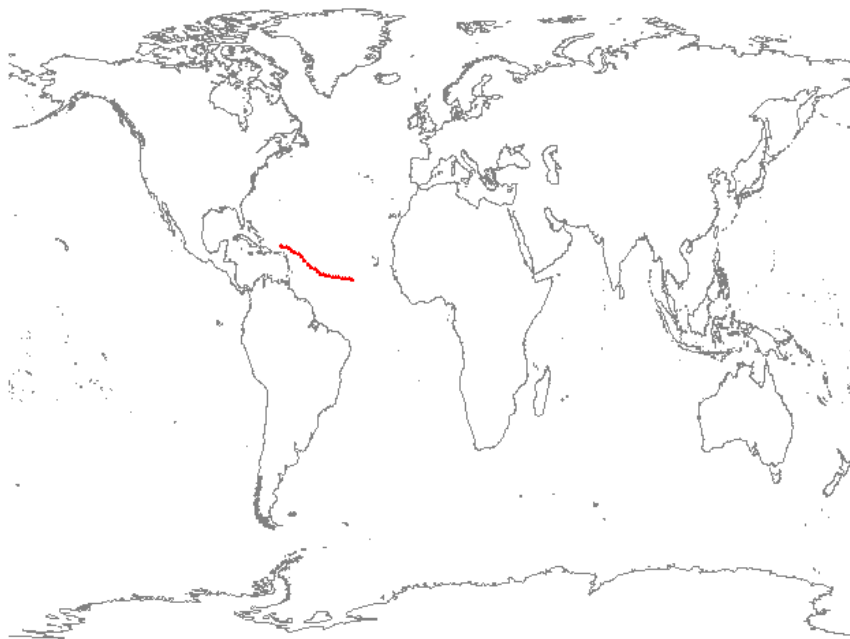


Figure 2: KML file: al112020.kml

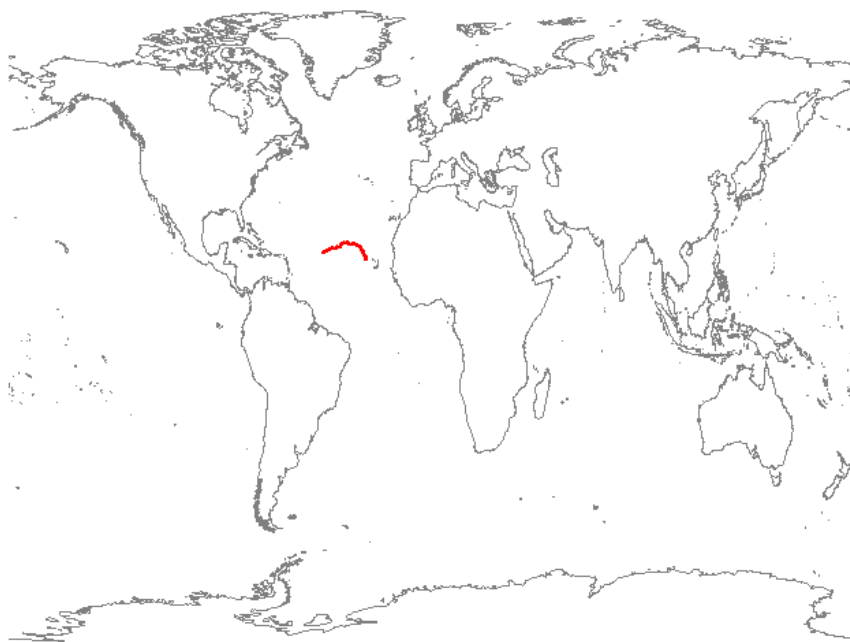


Figure 3: KML file: al212020.kml

4 Git usage

I ran the conflict-script.sh file that was provided with the coursework, which created a new branch named python-addon and a file named python-plot-script.py. This file was edited in both the python-addon branch and the working 'main' branch. I first made sure I was in my working 'main' branch by using "git checkout main". I merged with the python-addon branch by using the command "git merge python-addon" which created the conflict that I had to solve, as the python-plot-script.py files were different in both branches. I opened up the python-plot-script.py file with vim by using the command "vim python-plot-script.py" and I edited the file to solve the conflicts.

The first part of the script looked like this:

```
import pandas as pd
<<<<<<<< HEAD
import matplotlib.pyplot as plt
import os
import glob
import math
user_key = 12283
=====
import os
import glob
import matplotlib.pyplot as plt
user_key= 17647
>>>>>>> python-addon
```

The conflict here is mainly the fact that the user_key is different in both branches, I decided to keep the python-addon branch's part of the code here as the other branch imported 'math' which wasn't needed.

The second part of the script had this:

```
def plot_all_csv_pressure():
    path = os.getcwd()
    csv_files = glob.glob(os.path.join(path, '*.csv'))

<<<<<<<< HEAD
    for f in csv_files:
        storm = pd.read_csv(f)
        storm['Pressure'].plot()
        plt.show()
=====
    fr f in csv_files:
        storm = pd.read_csv(f)
        storm['Pressure'].plot()
        plt.show()

def plot_all_csv_intensity():
    path = os.getcwd()
    csv_files = glob.glob(os.path.join(path, '*.csv'))

    for f in csv_files:
        storm = pd.read_csv(f)
        storm['Intensity'].plot()
        plt.show()
>>>>>>> python-addon
```

The python-addon branch part of the code had more written in, with an entirely new function (plot_all_csv_intensity()) so I decided to keep that in, but the for loop had typos in so I kept the for loop from the main branch and removed the one from the python-addon branch.

Below is the python-plot-script.py script without any conflicts.

```
import pandas as pd
import os
import glob
import matplotlib.pyplot as plt
user_key = 17647

def plot_all_csv_pressure():
    path = os.getcwd()
    csv_files = glob.glob(os.path.join(path, '*.csv'))

    for f in csv_files:
        storm = pd.read_csv(f)
        storm['Pressure'].plot()
        plt.show()

def plot_all_csv_intensity():
    path = os.getcwd()
    csv_files = glob.glob(os.path.join(path, '*.csv'))

    for f in csv_files:
        storm = pd.read_csv(f)
        storm['Intensity'].plot()
        plt.show()
```